Serverless Event-Driven Application on AWS — Project Report

API Gateway · Lambda (Aliases & Stage Variables) · DynamoDB · IAM

Prepared as reference material for AWS Certified Developer – Associate (DVA-C02) preparation

AWS Account: 660235343407 | Region: us-east-1 | Date: July 1, 2026

Table of Contents

1. Executive Summary

This report documents the design, build, and troubleshooting of a serverless, event-driven backend on AWS. The system exposes a REST API via Amazon API Gateway, which routes traffic to an AWS Lambda function through environment-specific aliases selected using API Gateway Stage Variables. The Lambda function performs read operations against an Amazon DynamoDB table using the AWS SDK (boto3). The project was built hands-on and intentionally includes real misconfigurations and their resolutions, since debugging permission and routing issues is a core competency tested on the AWS Certified Developer – Associate (DVA-C02) exam.

The project touches four of the exam's core domains: Development with AWS Services, Security, Deployment, and Troubleshooting & Optimization. Each section below maps directly to relevant DVA-C02 exam objectives so this report can double as active-recall study material.

2. Architecture Overview

The application follows a standard three-tier serverless pattern:

```
Client
   |
   ▼
Amazon API Gateway (REST API, id: k9vv820z63)
   |  Stage: dev / prod   →   Stage Variable: env = dev | test
   ▼
AWS Lambda (function: lambda-event)
   |  Invoked via alias: lambda-event:${stageVariables.env}
   ▼
Amazon DynamoDB (table: LambdaEventTable)
```

The key architectural decision is that a single API Gateway deployment can invoke different Lambda code paths (aliases dev and test, which point to specific published Lambda versions) purely by changing a stage variable — no redeployment of the API or duplication of resources is required to support multiple environments.

2.1 Components

Component	Value	Purpose
API Gateway REST API	k9vv820z63	Public HTTPS entry point; routes GET / resource

Component	Value	Purpose
Stage Variable	env = dev, prod = test	Selects which Lambda alias is invoked at runtime
Lambda Function	lambda-event	Executes read logic against DynamoDB
Lambda Aliases	dev, test	Named, stable pointers to specific Lambda versions
IAM Execution Role	lambda-event-role-e581ks07	Grants the function permission to call DynamoDB & write logs
DynamoDB Table	LambdaEventTable	Persists event records

3. IAM & Security Configuration

Security in this project follows the principle of least-privilege execution roles, a heavily tested DVA-C02 concept (Domain 2: Security).

3.1 Execution Role

Role name: lambda-event-role-e581ks07

Role ARN: arn:aws:iam::660235343407:role/service-role/lambda-event-role-e581ks07

Trust policy: lambda.amazonaws.com (allows the Lambda service to assume this role)

Attached managed policies:

- **AWSLambdaBasicExecutionRole** (customer-managed copy) — grants logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents scoped to the function's log group.
- **AmazonDynamoDBFullAccess** (AWS managed) — grants broad DynamoDB permissions, added to resolve an AccessDeniedException encountered during testing (see Section 6).

Production note: AmazonDynamoDBFullAccess is intentionally over-privileged for a read-only Lambda. For a production system this should be replaced with a scoped, custom inline policy granting only dynamodb:GetItem, dynamodb:Query, and dynamodb:Scan on the specific table ARN — a distinction the DVA-C02 exam frequently tests (least privilege vs. managed convenience policies).

3.2 Resource-Based Policy (Lambda)

In addition to the execution role (identity-based policy, controls what the function can do), each invokable Lambda alias needs a resource-based policy (controls who can invoke the function). These were added explicitly via the CLI:

```
aws lambda add-permission   --function-name lambda-event:dev   --statement-id
apigw-dev   --action lambda:InvokeFunction   --principal
apigateway.amazonaws.com   --source-arn "arn:aws:execute-api:us-
east-1:660235343407:k9vv820z63/*/GET/resource"
```

This distinction — identity-based policy (attached to the role) vs. resource-based policy (attached to the Lambda resource itself, required for cross-service invocation from API Gateway) — is a frequently misunderstood DVA-C02 topic and was directly exercised in this project.

4. AWS Lambda Configuration

Function name: lambda-event

Function ARN: arn:aws:lambda:us-east-1:660235343407:function:lambda-event

Runtime: Python 3.x

Memory / **Timeout:** 128 MB (default configuration observed during testing)

4.1 Versions & Aliases

AWS Lambda versioning was used to decouple the mutable $LATEST code from stable, immutable snapshots that environments can safely point to:

- **Published Version 2** — a specific, immutable snapshot of the function code.
- **Alias dev** — points to a chosen published version for the development environment.
- **Alias test** — points to a chosen published version for the test/pre-prod environment.

Aliases allow the underlying version to be repointed (e.g. for a rollback or gradual rollout) without changing the API Gateway integration or any client-facing ARN, since the integration references the alias name, not the version number, via the stage variable substitution.

4.2 Read Logic (lambda_function.py)

The function supports four read patterns against DynamoDB, selected by which query string parameters are present on the incoming API Gateway event:

Request pattern	DynamoDB operation	Use case
?id=X&createdAt=Y	GetItem (full key)	Fetch one exact item — cheapest, fastest read
?id=X	GetItem (partition key only)	Only valid if the table has no sort key
?id=X&list=true	Query (partition key condition)	All items sharing one partition key
(no params)	Scan	Full table read — for small tables / dev only

Exam relevance: DVA-C02 explicitly tests the cost/performance tradeoffs between GetItem, Query, and Scan. GetItem and Query use the partition/sort key index directly and are efficient at any table size; Scan reads every item in the table and its cost/latency grows linearly with table size — it should never be used against a large production table on a hot code path.

5. API Gateway Configuration

API type: REST API

API ID: k9vv820z63

Method: GET /resource

Integration type: Lambda Proxy Integration

5.1 Stage Variables

Name	Value
env	dev
prod	test

The Lambda integration ARN uses a stage-variable placeholder instead of a hardcoded alias:

```
arn:aws:lambda:us-east-1:660235343407:function:lambda-event:$
{stageVariables.env}
```

This is the core technique that lets one API Gateway resource route to different Lambda code depending on which stage variable value is active — a pattern DVA-C02 covers under 'API Gateway deployment and stage management'.

6. DynamoDB Table Design

Attribute	Value
Table name	LambdaEventTable
Partition key	id (String)
Sort key	createdAt (String)
Billing mode	On-Demand (PAY_PER_REQUEST)
Sample item count	20 (loaded via BatchWriteItem)

A composite primary key (partition key + sort key) was chosen so that multiple events could share the same id while remaining individually addressable by createdAt, and so that Query (not just Scan) could retrieve all events for a given id efficiently — this models a common real-world event-log access pattern.

Bulk test data was loaded using BatchWriteItem rather than 20 individual PutItem calls:

```
aws dynamodb batch-write-item   --request-items file://batch-items.json   --
region us-east-1
```

Exam relevance: BatchWriteItem supports up to 25 items or 16 MB per call, does not support conditional writes, and can return UnprocessedItems under throttling that must be retried by the caller — all testable DVA-C02 details.

7. Testing Strategy

7.1 Lambda Console Test Events

Five JSON test events were authored to exercise every code path in isolation, directly in the Lambda console's Test tab, before wiring up API Gateway at all — isolating Lambda logic bugs from integration/permission bugs:

- get-item-by-id-and-sortkey.json — exact key lookup, expects 200
- get-item-by-id-only.json — partition-key-only lookup

- list-items-by-id.json — Query across a partition key
- scan-all-items.json — full table Scan
- get-item-not-found.json — expects a handled 404, not an unhandled exception

7.2 Integration Testing (End-to-End)

Beyond unit-level Lambda tests, integration tests were planned to validate the full path: HTTP request → API Gateway → Lambda alias → DynamoDB → HTTP response, run against both the dev and test stages by parameterizing the base URL and stage variable. This was scaffolded using pytest + boto3 + requests, with fixtures to seed and tear down DynamoDB test data per test run, avoiding test pollution.

Exam relevance: DVA-C02 Domain 4 (Troubleshooting and Optimization) expects familiarity with distinguishing a Lambda-layer failure from an API Gateway-layer failure from a downstream service (DynamoDB/IAM) failure — precisely what isolating unit tests from integration tests achieves.

8. Troubleshooting Log

Real issues encountered and resolved during the build are documented below, since debugging permission chains is one of the highest-yield DVA-C02 topics.

8.1 Issue: AccessDeniedException on dynamodb:Scan

Symptom: Lambda test invocation returned HTTP 500 with error: "User: arn:aws:sts::660235343407:assumed-role/lambda-event-role-e581ks07/lambda-event is not authorized to perform: dynamodb:Scan on resource: arn:aws:dynamodb:us-east-1:660235343407:table/LambdaEventTable because no identity-based policy allows the dynamodb:Scan action."

Root cause: The execution role initially lacked any DynamoDB permissions in its identity-based policy — only CloudWatch Logs permissions were present from AWSLambdaBasicExecutionRole.

Resolution: Attached the AmazonDynamoDBFullAccess managed policy to the role. Confirmed the fix using IAM Policy Simulator (aws iam simulate-principal-policy) rather than relying on trial-and-error re-invocation, since freshly attached IAM policies can take up to ~30-60 seconds to propagate across AWS's distributed authorization system — a subtlety that can look like a bug but is expected eventual-consistency behavior.

Exam takeaway: IAM is eventually consistent, not strongly consistent. The DVA-C02 exam tests awareness that a policy change is not guaranteed to be instantly visible everywhere, and that IAM Policy Simulator is the correct tool to validate a policy's effective permissions without needing to actually invoke the resource.

8.2 Issue: Resource-Based Policy Not Visible on Version Page

Symptom: After running aws lambda add-permission against lambda-event:dev and lambda-event:test, the Lambda console's Resource-based policy statements panel showed 'No policy statements.'

Root cause: The console was displaying the numbered Version (Version: 2) page, not the Alias page. Resource-based policies attached via a --function-name of functionName:aliasName are scoped to that alias object, which is distinct from any specific numbered version, even one the alias currently points to.

Resolution: Navigated to Lambda → Functions → lambda-event → Aliases → dev (and → test) to confirm the statements were present, and cross-verified with:

```
aws lambda get-policy --function-name lambda-event --qualifier dev
aws lambda get-policy --function-name lambda-event --qualifier test
```

Exam takeaway: DVA-C02 distinguishes Lambda $LATEST, numbered versions, and aliases as separate addressable entities, each of which can carry its own resource-based policy and environment variables. Confusing 'the alias' with 'the version it happens to point to' is a common source of 'my permissions look empty' confusion in the exam's scenario questions.

9. Mapping to AWS Certified Developer – Associate (DVA-C02) Domains

The DVA-C02 exam guide is organized into four domains. This project provided hands-on exposure to all four:

Domain	Weight	Concepts exercised in this project
1. Development with AWS Services	32%	Lambda function design, DynamoDB read patterns (GetItem/Query/Scan), event-driven Lambda handler structure, boto3 SDK usage
2. Security	26%	Identity-based vs. resource-based IAM policies, least-privilege role design, IAM policy simulation, execution role trust policy
3. Deployment	24%	Lambda versions & aliases, API Gateway stage variables for environment routing, BatchWriteItem for test-data provisioning
4. Troubleshooting and Optimization	18%	Diagnosing AccessDeniedException via CloudWatch logs, IAM eventual consistency, distinguishing alias vs. version scoping, isolating Lambda-layer vs. integration-layer failures

10. Key Learnings

- Stage variables let a single API Gateway deployment route to multiple Lambda aliases without duplicating infrastructure — critical for dev/test/prod parity.
- A Lambda function's permissions are governed by two separate policy types (identity-based on the role, resource-based on the function/alias), and both must be correct for a cross-service call chain to succeed.
- IAM changes are eventually consistent; don't assume a fresh AccessDeniedException after a policy fix means the fix was wrong — verify with the Policy Simulator before re-diagnosing.
- GetItem and Query should be preferred over Scan whenever the access pattern allows it, since Scan cost and latency scale with total table size, not result size.
- Separating Lambda-console unit tests from full end-to-end integration tests makes it much faster to localize whether a failure is in application logic, permissions, or routing.

11. Conclusion

This project implemented a working, multi-environment serverless read API and, in the process, surfaced two realistic AWS misconfigurations — a missing DynamoDB permission and a console navigation trap around Lambda alias vs. version resource policies. Both issues, and their resolutions, map directly onto tested DVA-C02 exam objectives around IAM policy types, eventual consistency, and Lambda's versioning model, making this build a practical complement to exam-prep study material.

Appendix A: Reference CLI Commands

```
# Create the DynamoDB table
aws dynamodb create-table   --table-name LambdaEventTable   --attribute-
definitions AttributeName=id,AttributeType=S
AttributeName=createdAt,AttributeType=S   --key-schema
AttributeName=id,KeyType=HASH AttributeName=createdAt,KeyType=RANGE   --billing-
mode PAY_PER_REQUEST --region us-east-1

# Load sample data
aws dynamodb batch-write-item --request-items file://batch-items.json --region
us-east-1

# Grant API Gateway permission to invoke a Lambda alias
aws lambda add-permission   --function-name lambda-event:dev   --statement-id
apigw-dev   --action lambda:InvokeFunction   --principal
apigateway.amazonaws.com   --source-arn "arn:aws:execute-api:us-
east-1:660235343407:k9vv820z63/*/GET/resource"

# Verify effective IAM permissions without invoking the resource
aws iam simulate-principal-policy   --policy-source-arn arn:aws:iam::
660235343407:role/service-role/lambda-event-role-e581ks07   --action-names
dynamodb:Scan   --resource-arns arn:aws:dynamodb:us-east-1:660235343407:table/
LambdaEventTable
```